



Amrita School of AI
COIMBATORE CAMPUS

25AID432 · MATHEMATICAL POSTULATES OF QUANTUM MECHANICS

Getting Started

Where the course material lives, how to set up Python and NumPy for checking your work, and how each part of the assignment is submitted.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

COURSE

25AID432 Mathematical Postulates of
Quantum Mechanics

REPOSITORY

github.com/Amrita-School-of-AI/mpqt-25aid432

READ THIS

In week one

TIME NEEDED

About thirty minutes, once

OFFERING

Odd Semester 2026-27

Read this once, in week one

Everything for this course lives in one public GitHub repository. This guide gets you from nothing to a working setup: the material downloaded, Python with NumPy running, and a clear picture of what you submit and when.

You do not need to know Git for this course. Downloading the PDFs from the web page is enough. If you already use Git, clone the repository instead and pull each week.

BY THE END OF THIS GUIDE YOU WILL HAVE

- The course repository open in a browser, and the handbook read.
- Python 3 and NumPy working, on your laptop or in Google Colab.
- Run your first check: a complex number calculation done by hand and confirmed in NumPy.
- A scanning app on your phone that produces one clean PDF from several pages.

Part 1: What is in the repository

github.com/Amrita-School-of-AI/mpqt-25aid432

Public. No login needed to read or download.

```
mpqt-25aid432/  
├─ handbook/      this guide, and the course handbook (plan, marks, policies)  
├─ notes/         three notes booklets, one per unit  
├─ slides/        the lecture decks, q01 to q08  
├─ assignments/   the Qubit Workbook: Parts A, B and C  
└─ exercises/     the Part C starter code and its self-check
```

BOOKLET	COVERS	SESSIONS
Unit 1: Complex Numbers	The complex plane, arithmetic, conjugate and modulus, Euler's formula, polar form, phase	1 to 3
Unit 2: Linear Algebra	Vectors and matrices over complex numbers, inner products, bases, projection, Hermitian and unitary matrices, eigenvalues, tensor products	4 to 9
Unit 3: The Postulates of Quantum Mechanics	Bra-ket notation, the qubit, measurement, operations, two qubits and entanglement, observables	10 to 15

To download a PDF: open it on the GitHub page, then use the download button at the top right of the file view.

NOTE · PRINT THE NOTES, OR KEEP THEM ON A TABLET

The notes are written to be worked through with a pen beside them. Reading them on a phone, one screen at a time, makes it hard to follow a calculation that runs over half a page.

Part 2: Python and NumPy

In this course NumPy has one job: to **check** a calculation you have already done by hand. You will not be asked to write large programs until Part C, and even there the program is about a hundred lines.

Pick **one** of the two options below.

Option A: Google Colab (nothing to install)

1. Go to `colab.research.google.com` and sign in with your college Google account.
2. Choose **New notebook**.
3. In the first cell, type the lines below and press **Shift + Enter**.

```
import numpy as np
print(np.__version__)
```

A version number means it works. Colab already has NumPy installed.

Option B: Python on your own laptop

Windows. Install Python 3 from `python.org/downloads`. On the first screen of the installer, tick **Add python.exe to PATH** before clicking Install. Then open **Command Prompt** and run:

```
py -m pip install numpy
py -c "import numpy; print(numpy.__version__)"
```

Ubuntu or Debian.

```
sudo apt install python3 python3-numpy
python3 -c "import numpy; print(numpy.__version__)"
```

macOS. Install Python 3 from `python.org/downloads`, then in Terminal:

```
python3 -m pip install numpy
python3 -c "import numpy; print(numpy.__version__)"
```

PITFALL · FORGETTING TO TICK ADD TO PATH ON WINDOWS

This is the most common setup problem. The symptom is that `py` works but `python` does not, or neither does. Re-run the installer, choose **Modify**, and tick the option. Using `py` instead of `python` also works on Windows.

Part 3: Your first check

Do this on paper first. Let $z = 3 + 4i$ and $w = 1 - 2i$. Work out:

1. $z + w$
2. zw
3. z^* , the complex conjugate of z
4. $|z|$, the modulus of z

Now check each answer. In Python the imaginary unit is written `1j`, with no space, because `i` and `j` are ordinary variable names.

```
import numpy as np

z = 3 + 4j
w = 1 - 2j
print(z + w)      # (4+2j)
print(z * w)      # (11-2j)
print(np.conj(z)) # (3-4j)
print(abs(z))     # 5.0
```

If a printed value disagrees with your paper, the paper is almost always the one that is wrong. Find the step where it went wrong; do not just copy the printed answer across. That habit is what the NumPy checks in Parts A and B are for.

PITFALL · `2j` IS A NUMBER, `2*j` IS A VARIABLE

`2j` means $2i$. `2*j` means “2 times whatever is stored in the variable `j`”, which is an error if there is none, and a silent wrong answer if there is one left over from a loop.

Part 4: How the marks work

The full scheme is in the handbook. In short:

COMPONENT	WEIGHT	WHAT IT IS
Class assignment	50%	The Qubit Workbook: Part A (10), Part B (14), Part C (20), viva (6)
End-semester examination	30%	Written paper on the whole syllabus
Class examination	20%	Multiple-choice test on Evolveus, Units 1 and 2

Parts A and B: handwritten, scanned

Parts A and B are problem sets done **by hand**, in a notebook or on sheets, with every step shown. When you finish:

1. Scan every page with a document-scanning app. Google Drive's **Scan** button and Microsoft Lens both work, and both produce one PDF from several pages with the edges cropped and the contrast fixed.
2. Check the pages are in order and every line is readable.
3. Name the file with your roll number and the part: `<ROLLN0>-partA.pdf` , `<ROLLN0>-partB.pdf` .
4. Submit the PDF, and the short `.py` file with your NumPy checks, through the channel announced in class.

PITFALL · A PHOTOGRAPH OF EACH PAGE IS NOT A SCAN

Ten separate photographs, taken at an angle, in a shadow, are hard to mark and easy to lose. Submit one PDF.

Part C: a qubit simulator

Part C is the one piece of real programming in the course. In `exercises/mpqt-ex01-qubit-simulator/` you get:

FILE	WHAT IT IS
<code>mpqt-ex01-qubit-simulator.pdf</code>	The statement: what each function must do, and how it is marked
<code>starter/qubit.py</code>	Every function named and documented, with an empty body
<code>tests/public.py</code>	The tests you can run yourself
<code>selfcheck.sh</code>	Runs all six stages of the tests in one go

Fill in the functions week by week as Unit 3 is taught. Then check your file. On Linux or macOS:

```
./selfcheck.sh qubit.py
```

On Windows or Colab, run one stage at a time. The stages are `states`, `probabilities`, `gates`, `measure`, `tensor` and `expectation`:

```
python tests/public.py qubit.py --stage states
```

Each test prints `ok`, or `FAIL:` with what was expected and what your code gave. Submit one file, `<ROLLNO>.py`. The grader runs the same stages with extra hidden tests on the same functions, so code that follows the docstrings for any input passes both.

Part 5: Rules

Late work. Marked out of 80 percent within 48 hours of the deadline; not accepted after that except with documented cause.

Your own work. Discuss approaches with anyone. Write your own answers and your own code.

AI tools. Permitted for learning: an explanation, another worked example, a hint. In the viva you will be asked to explain any step of Parts A and B and any line of Part C. What you cannot explain earns no marks.

Part 6: When something goes wrong

Bring three things: what you expected, what you got, and what you have already tried. For Python, bring the exact code and the exact error message, copied as text. For a calculation, bring the page of working.